

**Architecture
logicielle &
systèmes
distribués**[Accueil](#)[Introduction](#)[Jour 1](#)[Jour 2](#)[Jour 3](#)[Jour 4](#)[Jour 5](#)[Jour 6](#)[TP1](#)[TP2](#)[TP3](#)[Exercices](#)[Corrigés](#)[Glossaire](#)[Live Coding](#)[Dossier Architecture](#)[FAQ](#)[Support HTML modulaire](#)[Master 2](#)[6 jours](#)[3 TP progressifs](#)[Java / Spring Boot](#)

Jour 5 — Résilience, observabilité, sécurité & fiabilité

Objectifs pédagogiques

Fiabiliser un système distribué : résilience
Resilience4j, observabilité
3 pillars (ELK +
Prometheus + Jaeger),
sécurité JWT/OAuth2 et
Zero Trust.

Temps estimé

≈ 6h incluant démos, live
coding et quiz

Prérequis

Jours 1 à 4 acquis —
microservices,
communication inter-
services

Table des matières

- 1. Résilience
 - — Timeout & Retry
 - — Circuit Breaker
 - — Bulkhead & Rate Limiter
- 2. Observabilité
 - — Logs structurés
 - — Métriques Micrometer
 - — Traces distribuées
- 3. Health Checks
- 4. Sécurité JWT / OAuth2
- 5. Zero Trust & Secrets

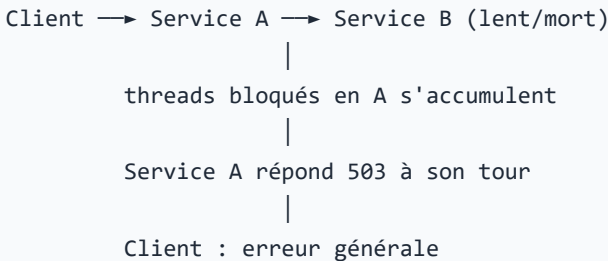
- [Quiz de fin](#)

1. Résilience : empêcher la panne locale de devenir globale

🧒 Mode enfant

Imagine que la cantine ne réponde plus. Sans résilience, tous les élèves restent bloqués à attendre devant la porte fermée. Avec résilience, la cantine propose une alternative (fallback) ou dit "reviens dans 10 minutes" (circuit breaker). L'école continue de fonctionner.

Dans un système distribué, un seul service lent peut provoquer une **cascade failure** : ses appelants s'entassent sur des threads bloqués, ce qui bloque leurs propres appelants, et ainsi de suite jusqu'à l'indisponibilité totale.



Mécanisme	Rôle	Analogie	Config typique
Timeout	Couper l'attente excessive	Minuterie de four	500ms – 2s
Retry	Retenter un échec transitoire	Rappeler après une mauvaise ligne	2–3 fois, backoff expo
Circuit Breaker	Stopper les appels vers un service malade	Disjoncteur électrique	Ouvrir après 50% d'échecs
Fallback	Réponse dégradée mais utile	Mode hors-ligne d'un GPS	Cache, valeur par défaut
Bulkhead	Isoler les ressources par service	Compartiments d'un sous-marin	Thread pool séparé
Rate Limiter	Limiter le débit entrant	Péage d'autoroute	100 req/s max

1.1 Timeout & Retry

● Junior

Le timeout abandonne l'attente d'une réponse après un délai défini. Le retry retente automatiquement en cas d'échec *transitoire*. **Ne jamais configurer un retry sans timeout.**

```
APPLICATION.YML — RESILIENCE4J

resilience4j:
  timelimiter:
    instances:
      catalogService:
        timeout-duration: 1500ms
        cancel-running-future: true

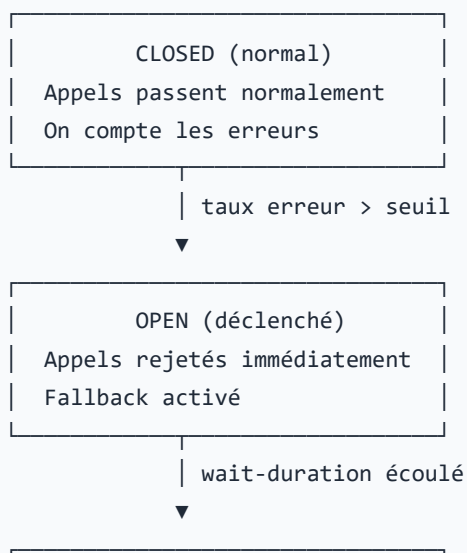
  retry:
    instances:
      catalogService:
        max-attempts: 3
        wait-duration: 200ms
        enable-exponential-backoff: true # 200ms, 400ms, 800ms
        exponential-backoff-multiplier: 2
        # Ne PAS retenter Les erreurs métier (404, 400)
        ignore-exceptions:
          - com.example.exception.BookNotFoundException
```

⚠ Idempotence obligatoire avant tout retry

Ne retenter que les opérations **idempotentes** (GET, PUT, DELETE). Un retry sur un POST de création peut créer plusieurs fois la même ressource. Toujours vérifier avant d'activer le retry sur une opération d'écriture.

1.2 Circuit Breaker — Les 3 états

● Intermédiaire



HALF-OPEN (sonde)
X appels de test autorisés
Succès → CLOSED, Échec → OPEN

• • • APPLICATION.YML — CIRCUIT BREAKER COMPLET

```
resilience4j:
  circuitbreaker:
    instances:
      catalogService:
        sliding-window-type: COUNT_BASED
        sliding-window-size: 10
        failure-rate-threshold: 50           # Ouvrir si 50% d'échecs
        slow-call-rate-threshold: 80        # Appels lents > seuil aussi = échec
        slow-call-duration-threshold: 1000ms
        wait-duration-in-open-state: 30s
        permitted-number-of-calls-in-half-open-state: 3
        register-health-indicator: true
```

• • • LOANSERVICE.JAVA — ANNOTATIONS COMBINÉES

```
@Service
@Slf4j
public class LoanService {

    // Ordre d'exécution (externe → interne) :
    // TimeLimiter → CircuitBreaker → Retry
    @TimeLimiter(name = "catalogService")
    @CircuitBreaker(name = "catalogService",
        fallbackMethod = "fallbackAvailability")
    @Retry(name = "catalogService")
    public CompletableFuture<AvailabilityResponse> getAvailability(UUID bookId) {
        return CompletableFuture.supplyAsync(
            () -> catalogClient.getAvailability(bookId)
        );
    }

    // Même signature + Throwable en dernier
    private CompletableFuture<AvailabilityResponse> fallbackAvailability(
        UUID bookId, Throwable ex) {
        log.warn("Fallback triggered bookId={} cause={}", bookId, ex.getMessage());
        return CompletableFuture.completedFuture(
            AvailabilityResponse.degraded(bookId)
        );
    }
}
```

1.3 Bulkhead & Rate Limiter

 Expert

Le Bulkhead isole les ressources : un service externe lent ne monopolise pas **tous** les threads. Deux modes possibles :

Sémaphore (même thread, limite de concurrence)

```
resilience4j:
  bulkhead:
    instances:
      catalogService:
        max-concurrent-calls: 10
        max-wait-duration: 10ms
```

Thread Pool (thread dédié, isolation totale)

```
resilience4j:
  thread-pool-bulkhead:
    instances:
      paymentService:
        core-thread-pool-size: 4
        max-thread-pool-size: 8
        queue-capacity: 20
```

✓ Ordre recommandé des patterns

De l'externe vers l'interne : `Retry → CircuitBreaker → RateLimiter → TimeLimiter → Bulkhead`.

Jamais Retry à l'intérieur du CircuitBreaker : les retries gonfleraient artificiellement la fenêtre d'erreurs et ouvriraient le circuit trop tôt.

Live Coding — Cascade failure

1. Démarrer un `slow-service` répondant après 3 secondes
2. Appeler depuis `loan-service` sans timeout → observer les threads bloqués via `/actuator/metrics/jvm.threads.live`
3. Activer le TimeLimiter → les appels reviennent avec fallback en 1,5 s

4. Activer le CircuitBreaker → après 5 échecs, les appels sont court-circuités immédiatement

2. Observabilité : les 3 piliers

L'observabilité répond à trois questions fondamentales en production :

Logs

Quoi s'est passé, quand, dans quel contexte. Événements discrets horodatés.

Logback → Elasticsearch → Kibana (ELK)

Métriques

Combien d'événements, quelle latence, quel taux d'erreur. Agrégats dans le temps.

Micrometer → Prometheus → Grafana

Traces

Où passe la requête entre services. Parcours complet d'une transaction.

OpenTelemetry → Jaeger / Tempo / Zipkin

2.1 Logs structurés et corrélation MDC

● Junior

MDC — Mapped Diagnostic Context

Dictionnaire thread-local de SLF4J. Chaque entrée est automatiquement ajoutée à **tous les logs** du thread courant, sans avoir à passer les métadonnées explicitement à chaque méthode.

✗ LOG SANS CONTEXTE

• • • JAVA

```
// Impossible de retrouver tous les logs d'un emprunt précis
log.info("Emprunt créé");
log.error("Erreur lors de la création");
```

✓ LOG STRUCTURÉ AVEC CORRÉLATION

• • • LOANCONTROLLER.JAVA

```
@RestController
@Slf4j
public class LoanController {

    @PostMapping("/api/loans")
    public ResponseEntity<LoanDto> createLoan(
        @RequestBody CreateLoanRequest req,
        @RequestHeader("X-Correlation-Id") String correlationId) {

        MDC.put("correlationId", correlationId);
        MDC.put("studentId", req.getStudentId().toString());

        try {
            LoanDto loan = loanService.createLoan(req);
            // correlationId et studentId apparaissent dans le log automatiquement
            log.info("loan-created loanId={} bookId={}", loan.getId(), loan.getBookId());
            return ResponseEntity.status(201).body(loan);
        } finally {
            MDC.clear(); // TOUJOURS nettoyer pour éviter les fuites
        }
    }
}
```

• • • LOGBACK-SPRING.XML — FORMAT JSON POUR ELK

```
<configuration>
  <appender name="JSON" class="ch.qos.logback.core.ConsoleAppender">
    <encoder class="net.logstash.logback.encoder.LogstashEncoder">
      <includeMdcKeyName>correlationId</includeMdcKeyName>
      <includeMdcKeyName>studentId</includeMdcKeyName>
      <customFields>{"service":"loan-service","env":"prod"}</customFields>
    </encoder>
  </appender>
  <root level="INFO"><appender-ref ref="JSON"/></root>
</configuration>
```

2.2 Métriques avec Micrometer

● Intermédiaire

Type	Usage	Exemple
Counter	Événements monotones (compteur)	Prêts créés

Type	Usage	Exemple
Gauge	Valeur instantanée variable	Emprunts en cours
Timer	Durée + nombre d'opérations	Latence appel HTTP
DistributionSummary	Distribution de valeurs	Taille des payloads

• • • LOANSERVICE.JAVA — MÉTRIQUES MÉTIER

```
@Service
public class LoanService {

    private final Counter loansCreated;
    private final Counter loansRejected;
    private final Timer loanCreationTimer;

    public LoanService(MeterRegistry registry) {
        loansCreated = Counter.builder("loans.created")
            .description("Prêts créés avec succès")
            .register(registry);
        loansRejected = Counter.builder("loans.rejected")
            .description("Prêts refusés (quota, indisponibilité)")
            .register(registry);
        loanCreationTimer = Timer.builder("loans.creation.duration")
            .description("Durée de création d'un prêt")
            .publishPercentiles(0.5, 0.95, 0.99) // p50, p95, p99
            .register(registry);
    }

    public LoanDto createLoan(CreateLoanRequest req) {
        return loanCreationTimer.record(() -> {
            validateQuota(req.getStudentId());
            LoanDto loan = LoanMapper.toDto(loanRepository.save(toLoan(req)));
            loansCreated.increment();
            return loan;
        });
    }
}
```

2.3 Traces distribuées avec OpenTelemetry

● Expert

```
POST /api/loans (200ms total)
├─ [loan-service]      span: validate-quota      (5ms)
```

```
└─ [catalog-service] span: GET /availability (80ms) ← goulot !
└─ [notif-service] span: POST /notify (15ms async)
```

Un **span** est une unité de travail. Une **trace** est l'arbre de spans partagés via un même **traceId**, propagé dans le header HTTP **traceparent** (standard W3C Trace Context).

• • • POM.XML — OPENTELEMETRY + ZIPKIN

```
<dependency>
  <groupId>io.micrometer</groupId>
  <artifactId>micrometer-tracing-bridge-otel</artifactId>
</dependency>
<dependency>
  <groupId>io.opentelemetry</groupId>
  <artifactId>opentelemetry-exporter-zipkin</artifactId>
</dependency>
```

• • • APPLICATION.YML — TRACING

```
management:
  tracing:
    sampling:
      probability: 1.0 # 100% en dev, 0.1 en prod (10%)
spring:
  zipkin:
    base-url: http://zipkin:9411
# Le traceId est automatiquement injecté dans Les Logs MDC
```

• • • LOANSERVICE.JAVA — SPAN PERSONNALISÉ

```
@Service
public class LoanService {

    @Autowired Tracer tracer;

    public LoanDto createLoan(CreateLoanRequest req) {
        Span span = tracer.nextSpan()
            .name("loan.quota-check")
            .tag("studentId", req.getStudentId().toString())
            .start();

        try (Tracer.SpanInScope ws = tracer.withSpan(span)) {
            validateQuota(req.getStudentId());
            return doCreateLoan(req);
        } catch (QuotaExceededException e) {
            span.tag("error", "quota-exceeded");
            throw e;
        }
    }
}
```

```

    } finally {
        span.end(); // TOUJOURS finir Le span
    }
}
}

```

✓ Corrélation logs + traces

Micrometer Tracing injecte automatiquement `traceId` et `spanId` dans le MDC Logback. Résultat : chaque ligne de log contient le `traceId`, permettant de passer de Kibana (logs) à Grafana Tempo (traces) d'un seul clic.

3. Monitoring, Health Checks & alerting

👤 Mode enfant

Un service peut "tourner" (le process est vivant) mais être incapable de répondre (la base de données est coupée). Le health check c'est comme appuyer sur un bouton de test réel — il vérifie si tout fonctionne en profondeur, pas juste si le moteur tourne.

Sonde	Question	Réponse KO	Action Kubernetes
Liveness	Le process est-il vivant ?	Deadlock, corruption mémoire	Redémarre le pod
Readiness	Peut-il recevoir du trafic ?	DB coupée, warm-up en cours	Retire du load balancer
Startup	A-t-il fini de démarrer ?	Init longue (> 30s)	Attend avant les autres sondes

• • • APPLICATION.YML — ACTUATOR COMPLET

```

management:
  endpoints:
    web:
      exposure:
        include: health,info,prometheus,metrics,circuitbreakers
  endpoint:
    health:
      probes:
        enabled: true           # Active /actuator/health/liveness et /readiness

```

```

show-details: always    # Détail des composants (à restreindre en prod)
metrics:
  export:
    prometheus:
      enabled: true
  tags:
    application: loan-service
    env: ${SPRING_PROFILES_ACTIVE:dev}

```

● Intermédiaire

• • • CATALOGSERVICEHEALTHINDICATOR.JAVA

```

@Component
public class CatalogServiceHealthIndicator implements HealthIndicator {

    private final CatalogClient catalogClient;
    private final CircuitBreakerRegistry cbRegistry;

    @Override
    public Health health() {
        CircuitBreaker cb = cbRegistry.circuitBreaker("catalogService");
        if (cb.getState() == CircuitBreaker.State.OPEN) {
            return Health.down()
                .withDetail("circuit-breaker", "OPEN")
                .withDetail("reason", "Trop d'échecs vers catalog-service")
                .build();
        }
        try {
            catalogClient.ping();
            return Health.up()
                .withDetail("circuit-breaker", cb.getState().name())
                .build();
        } catch (Exception e) {
            return Health.down().withDetail("error", e.getMessage()).build();
        }
    }
}

```

✓ 4 Golden Signals (Google SRE)

- **Latency** — Temps de traitement (p95, p99)
- **Traffic** — Volume de requêtes (req/s)
- **Errors** — Taux d'erreurs 5xx
- **Saturation** — CPU, mémoire, threads, connexions DB

Une alerte Prometheus sur chacun de ces 4 indicateurs est le minimum viable.

● Expert

```
PROMETHEUS ALERT RULE - TAUX D'ERREUR

- alert: HighErrorRate
  expr: |
    rate(http_server_requests_seconds_count{status=~"5.."}[5m])
    /
    rate(http_server_requests_seconds_count[5m]) > 0.05
  for: 2m
  labels:
    severity: critical
  annotations:
    summary: "Taux d'erreur élevé sur {{ $labels.application }}"
    description: "{{ $value | humanizePercentage }}" d'erreurs 5xx détectées"
```

4. Sécurité inter-services : JWT, OAuth2 / OIDC, RBAC

🧒 Mode enfant

Le badge d'entrée (JWT) dit *qui tu es* et *ce que tu peux faire*. Authentification = prouver son identité.
Autorisation = vérifier ses droits.

4.1 Anatomie d'un JWT

● Junior

Un JWT est signé en **3 parties séparées par des points** : `header.payload.signature`. Chaque partie est encodée en Base64URL — lisible, **pas chiffré**.

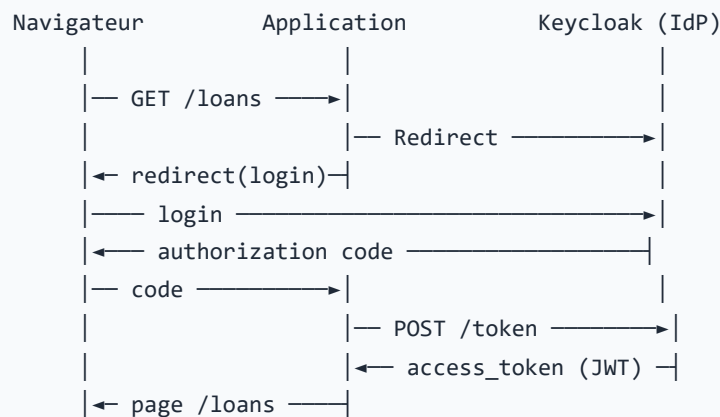
```
eyJhbGciOiJSUzI1NiJ9 . eyJzdWIiOiJ1c2VyMTIzIn0 . abc123sig...
  ▲ Header                ▲ Payload (claims)                ▲ Signature RSA
{ alg: "RS256" }          { sub: "user123",                Vérifiée avec la clé
{ typ: "JWT" }             roles: ["LIBRARIAN"],          publique de l'IdP
                           exp: 1735686000 }
```

⚠️ JWT ≠ chiffrement

Le payload est lisible par n'importe qui. Ne jamais y mettre de données sensibles. La signature garantit l'intégrité, pas la confidentialité.

4.2 Flux OAuth2 Authorization Code (OIDC)

Intermédiaire



SECURITYCONFIG.JAVA — RESOURCE SERVER

```

@Configuration
@EnableWebSecurity
@EnableMethodSecurity
public class SecurityConfig {

    @Bean
    SecurityFilterChain filterChain(HttpSecurity http) throws Exception {
        return http
            .csrf(csrf -> csrf.disable()) // API stateless = pas de CSRF
            .sessionManagement(s ->
                s.sessionCreationPolicy(STATELESS))
            .authorizeHttpRequests(auth -> auth
                .requestMatchers("/actuator/health/**").permitAll()
                .requestMatchers(GET, "/api/books/**").hasAnyRole("USER", "LIBRARIAN")
                .requestMatchers(POST, "/api/loans/**").hasRole("LIBRARIAN")
                .anyRequest().authenticated()
            )
            .oauth2ResourceServer(oauth ->
                oauth.jwt(jwt -> jwt.jwtAuthenticationConverter(
                    jwtAuthenticationConverter()))
            )
            .build();
    }

    @Bean
    JwtAuthenticationConverter jwtAuthenticationConverter() {
  
```

```

    JwtGrantedAuthoritiesConverter conv = new JwtGrantedAuthoritiesConverter();
    conv.setAuthoritiesClaimName("roles"); // claim custom dans Keycloak
    conv.setAuthorityPrefix("ROLE_");
    JwtAuthenticationConverter jwtConv = new JwtAuthenticationConverter();
    jwtConv.setJwtGrantedAuthoritiesConverter(conv);
    return jwtConv;
}
}

```

• • • LOANCONTROLLER.JAVA — @PREAUTHORIZE FIN

```

@RestController
@RequestMapping("/api/loans")
public class LoanController {

    // L'étudiant ne voit QUE ses propres emprunts
    @GetMapping("/{loanId}")
    @PreAuthorize(""""
        hasRole('LIBRARIAN') or
        @loanService.isOwner(#loanId, authentication.name)
        """)
    public LoanDto getLoan(@PathVariable UUID loanId) {
        return loanService.findById(loanId);
    }

    @PostMapping
    @PreAuthorize("hasRole('LIBRARIAN')")
    public ResponseEntity<LoanDto> createLoan(
        @RequestBody @Valid CreateLoanRequest req) {
        return ResponseEntity.status(201).body(loanService.createLoan(req));
    }
}

```

4.3 mTLS — Authentification mutuelle inter-services

● Expert

Mode pro — JWT vs mTLS

- **JWT** : identité de l'utilisateur, propagée dans le header `Authorization`. Validé au niveau applicatif.
- **mTLS** : identité du service, négociée au niveau TLS. Validé par le service mesh (Istio, Linkerd), transparent pour l'application.
- En production on combine les deux : mTLS pour le service-to-service + JWT pour le contexte utilisateur.

✖ Erreur fréquente — Valider le JWT soi-même

Ne jamais implémenter la validation JWT manuellement. Erreurs classiques : oublier de vérifier la signature, ne pas vérifier `exp`, accepter l'algorithme `none`. Utiliser une bibliothèque maintenue : `spring-security-oauth2-resource-server` ou `nimbus-jose-jwt`.

5. Zero Trust & gestion des secrets

Zero Trust — "Never trust, always verify"

Aucun segment réseau n'est implicitement fiable, même interne. Chaque accès sensible est vérifié : identité, contexte, droits minimaux. Un attaquant ayant pénétré le périmètre ne peut pas se déplacer librement.

5.1 Les 3 règles fondamentales des secrets

● Junior

✖ NE JAMAIS FAIRE — SECRET DANS GIT

```
• • • APPLICATION.YML VERSIONNÉ

spring:
  datasource:
    password: MonSuperMotDePasse123! # visible dans l'historique Git pour toujours
  security:
    oauth2:
      client:
        registration:
          keycloak:
            client-secret: abc-def-secret # même problème
```

✓ VARIABLES D'ENVIRONNEMENT OU VAULT

```
• • • APPLICATION.YML SÉCURISÉ

spring:
  datasource:
    url:      ${DATABASE_URL}
    username: ${DATABASE_USER}
    password: ${DATABASE_PASSWORD}
  security:
    oauth2:
```

```
client:
  registration:
    keycloak:
      client-secret: ${KEYCLOAK_CLIENT_SECRET}
```

5.2 HashiCorp Vault avec Spring Cloud Vault

● Intermédiaire

```
• • • BOOTSTRAP.YML — VAULT + AUTH KUBERNETES

spring:
  cloud:
    vault:
      host: vault.internal.company.com
      port: 8200
      scheme: https
      authentication: KUBERNETES
      kubernetes:
        role: loan-service
        service-account-token-file: /var/run/secrets/kubernetes.io/serviceaccount/token
      kv:
        enabled: true
        backend: secret
        default-context: loan-service
```

5.3 Dynamic Secrets — Credentials DB éphémères

● Expert

Vault génère des credentials PostgreSQL **à la demande avec une TTL courte** (ex. 1h). L'application ne connaît jamais le mot de passe permanent. En cas de compromission, le credential expire automatiquement.

```
• • • BOOTSTRAP.YML — DYNAMIC SECRETS POSTGRESQL

spring:
  cloud:
    vault:
      database:
        enabled: true
        role: loan-service-role
        backend: database
        # Vault renouvelle automatiquement le lease avant expiration
```

✓ Checklist Sécurité Production

- Aucun secret dans Git (même un commit "révoqué" reste dans l'historique)
- Rotation des clés JWT tous les 90 jours minimum
- Moindre privilège : chaque service accède uniquement à ses secrets
- Audit des accès aux secrets (Vault fournit un audit log)
- mTLS en production via service mesh (Istio / Linkerd)
- Scan des images Docker pour les CVE (`trivy` , `grype`)

Quiz de fin — Jour 5

1. Pourquoi un retry sans timeout peut-il être dangereux ?

► [Voir la réponse](#)

2. Quelle différence entre logs, métriques et traces ?

► [Voir la réponse](#)

3. Quelle différence entre liveness et readiness probe ?

► [Voir la réponse](#)

4. Pourquoi le réseau interne n'est pas sûr par défaut ? (Zero Trust)

► [Voir la réponse](#)

5. **(Expert)** Pourquoi ne pas mettre le Retry à l'intérieur du Circuit Breaker ?

► [Voir la réponse](#)

6. **(Expert)** Quelle propriété garantit qu'un JWT n'a pas été modifié en transit ?

► [Voir la réponse](#)

[← Jour 4](#)

[Jour 6 →](#)